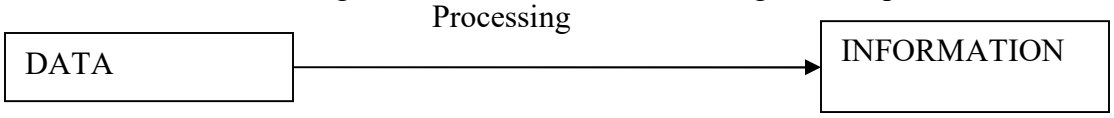
B.Sc(MATHS) - IV SEMESTER- DBMS

Q.1. Define Data, Information, Metadata. Compare them. Give example.

Ans. **DATA**: Data is a raw collection of characters, facts about objects. Data includes text, graphics, images, sounds and etc.

INFORMATION: Information is meaningful data in an organized form. Information is processed data that increases the knowledge of the user.

In short, Data is meaning less and Information is meaning full and processed data.



META DATA: Meta data is data about data. It describes the properties or characteristics of other data. It includes field names, data-types and their size.

Data	Meta data	Information
Data is a raw collection of characters, facts	Meta data is data about data	Inforamtion is meaningful data in an organized form.
Data exists with the user	Meta data is required for entering data	Information is required by the user.
It is unprocessed data	It requires defining using languages such as SQL	This is processed data
Required for database analysis	Required for database design	Required for decision making

Ravi 33 88 77 94 This is DATA

Meta data →

Name (text)	Rno (number)	M1 (number)	M2 (number)	M3 (number)	Total (number)
Ravi	33	88	77	94	
Srinu	47	82	54	87	

Q.2. What are the limitations of manual data processing?

Ans. Manual Data Processing: In this, the processing is done by the human beings. As all the calculations are done by the human beings, there is chance of having errors, which leads to poor processing. This process is very slow and imparctical in the case of large amount of data.

The Limitations of Manual of data processing:

1. **Less Processing Speed:** Manual Data processing consumes lots of time, even for small amounts of work. This process is very slow and impractical in the case of lage amounts of data.
2. **Less Accuracy:** The data processed by the human beings is less accurate. Errors are very huge in case of complex calculations.
3. **Storage problems:** Large volumes of data cannot be conveniently stored and accessed. All the data has to be stored in the form of documents, which may get torn or damaged easily.
4. **Difficult to Manage Information:** We cannot easily access the required information and prepare reports for managerial decision-making.
5. **Cost overheads:** Increase in cost occurs due to processing of huge data and record keeping and the need of more workforce.
6. **Diligence:** Macines do not suffer from boredom, tiredness, or lack of concentration, evenif millions of calculations are to be performed. But this is not so in the case of manual processing as it is done by human beings. One cannot work for hours together as computers do.

7. **No Back up and Recovery:** There is nothing like backup and recovery in the case of manual data procssing. The only way to backup is to keep multiple copies of same documents, which again requires machines like copiers.

Q.3 What are the Disadvantages of File Oriented system? OR
What are the Limitations of Traditional File Systems?

Ans. **Disadvantages of File Processing System:** File Processing System was first to replace non-computer based approach for maintaining records. It was a successful System of its time and still there are many organizations that are using File Processing System to maintain their data and information. But it is just not suitable for handling data of big firms and organizations. It has many drawbacks and disadvantages that made it out of date.

Now as we know that File Processing System uses different files to store data. So it created lots of disadvantages in File Processing System.

Disadvantages of File Processing System

Duplicate Data: Data is stored more than once in different files, that means duplicate data may occur in all these files. Since all the files are independent on each other so it is very difficult to overcome this error and if anyone finds this error then it will take time and effort to solve this issue.

For Example: A student is having record in college library and in Examination department. Then his name, roll number, fathers name and class will be same in both the departments. Also these departments are not dependent on each other. So it create lots of duplicates value about that student and when he needs any change for his name or class then he has to go to both the departments to make these changes happen otherwise it will create problem for him.

Inconsistency: In file processing system, various copies of same data may contain different values. Data is not consistent in this system, it means if a data item needs to be changed then all the files containing that data need to be modified. It may create a risk of out dated values of data.

For Example: If you change student name in library then his name should be changed in all the departments related to the student.

Accessing Anomalies: Accessing anomalies means that it is not easy to access data in a desired or efficient way. It makes supervision of department very difficult. If a user wants information in a specific manner then he requires creating a program for it.

For Example: Let's say , if admin of the college wants any student information like his name, fathers name, roll number, marks and class then program for it is written but if he wants records of whose students whose numbers are more than 80 percent then he require to create a different program for it.

Poor Data Integrity: A collection of data is integrated if it meets certain consistency constraints. A programmer always puts these constraints in the programs by adding some codes. In File Processing System, poor data integrity often arises and it becomes very difficult to add new constraints at that time.

For Example: The maximum marks of the student can never be more than 100.

Poor Data Security: Poor data security is the most threatening problem in File Processing System. There is very less security in File Processing System as anyone can easily modify and change the data stored in the files. All the users must have some restriction of accessing data up to a level.

For Example: If a student can access his data in the college library then he can easily change books issued date. Also he can change his fine detains to zero.

Atomicity Problem: Atomicity is required to save the data values, it means that information is completely entered or canceled at all. Any system may fail at any time and at that time it is desired that data should be in a consistent state.

For Example: If you are buying a ticket from railway and you are in the process of money transaction. Suddenly, your internet got disconnected then you may or may not have paid for the ticket. If you have paid then your ticket will be booked and if not then you will not be
charged anything. That is called consistent state, means you have paid or not. Same atomicity is not present in File Processing System.

Wastage of Labor and Space: Labor is very costly in this era and no organization can afford wastage of their precious labor. File Processing System needs lots of copied data in different files that cause

wastage of labor. Also maintaining same data again and again leads to wastage of space too. For Example: Maintaining student's record in many departments that are not dependent on each other cause wastage of labor and space.

Data Isolation:

Data is isolated in File Processing System and data is stored in different files. These files can be in different formats. If you want to extract data from two file then you are required to which part of the file is needed and how they are related to each other.

But still in spite of so many disadvantages, File Processing System is still good for small organizations because it does not require costly software's and programmers to handle

Q.4. Define DBMS. Explain its advantages (purpose of database system) over traditional file management system.

Ans. "A Data base is a collection of logically related data stored in a standardized format and is sharable by multiple users." **DBMS (Data Base management System)** is collection of database and a set of application programs that allow us to organize, manipulate and retrieve information.

Advantages of DBMS:

- **Improved data sharing:** A database is designed as a shared resource. Authorized users are granted permission to use the database. Data sharing is a primary feature of a Database management system.
- **Improved data security:** When number of users increases to access the data, the risk of data security increases. But, DBMS provides a framework for better enforcement of data privacy and security policies. A database can be accessed only by proper authentication usually by verifying login and password
- **Minimized data inconsistency:** Data inconsistency exists when different versions of the same data appear in different places. In a DBMS, by eliminating this data redundancy, we can improve data consistency. For example, if a customer address is stored only once, updating that becomes simple.
- **Improved data access:** The DBMS makes it possible to produce quick answers to any queries. **A query is a request or a question put to the DBMS for data manipulation or retrieval**. Without any programming experience, one can retrieve and display data very easily. The language used to write queries is called Structured Query Language (SQL).
- **Minimal Data Redundancy:** Unnecessary data can be reduced. There will not be any duplicate records. Each primary fact is recorded in only one place in the database.
- **Restricting unauthorized access:** When multiple users share a database, it is likely that some users will not be authorized to access all information in the database. For example, account office data is often considered confidential, and hence, only authorized persons are allowed to access such data. In addition, some users may be permitted only to retrieve data, whereas others are allowed both to retrieve and to update.
- **Focus on Data:** DBMS focuses on data. It first defines the data and then all queries, reports and programs to access the data through the DBMS. The DBMS is responsible for defining, storing and retrieving the data**.**
- **Backup and recovery:** A DBMS must provide facilities for recovering from hardware of software failures. The backup and recovery subsystem of the DBMS is responsible for recovery. For example, if the computer system fails in the middle of a complex update program, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the program started executing.

Q: Explain Database system Applications.

Database system applications are found in almost every field where large amounts of data need to be stored, managed, and retrieved efficiently. Some important applications are:

1. Banking Systems:

- Used to store customer details, accounts, transactions, loans, and ATM records.
- Ensures accuracy, security, and fast access to data.

2. Airline & Railway Reservation Systems:

- Manage reservations, ticket booking, cancellations, schedules, and passenger details.
- Databases help handle thousands of users at the same time.

3. Education Systems:

- Used in schools and colleges to store student information, marks, attendance, timetables, and exam results.
- Example: Student Management Systems.

4. Hospital & Healthcare Systems:

- Maintain patient records, doctor details, appointments, medical history, and billing.
- Helps in quick and reliable access to patient data.

5. E-Commerce Applications:

- Used by online shopping platforms to manage products, customers, orders, payments, and delivery details.
- Example: Amazon, Flip kart.

6. Library Management Systems:

- Store book details, issue/return records, fines, and member information.
- Reduces manual work and improves efficiency.

7. Telecommunications:

- Manage customer data, call records, billing information, and network operations.
- Used by mobile and internet service providers.

8. Government Applications:

- Used for census data, voter lists, tax records, Aadhaar, land records, and public services.

9. Payroll & Human Resource Systems:

- Store employee details, salaries, attendance, leaves, and performance data.

10. Social Media Applications:

- Store user profiles, posts, messages, likes, and comments.
- Example: Face book, Instagram, Twitter.

Q.5 Explain the various type of Record based logical models.

Ans. Record-based logical models describe data at athe conceptual and view levels. Unlike object oriented models, they are used to specify overall logical structure of the database. The structure of these databases is in fixed format records. Each record type defines a fixed number of fields, or attributes. Record based models do not include a mechanism or direct representation of code in the database. The three most widely accepted record based models are :

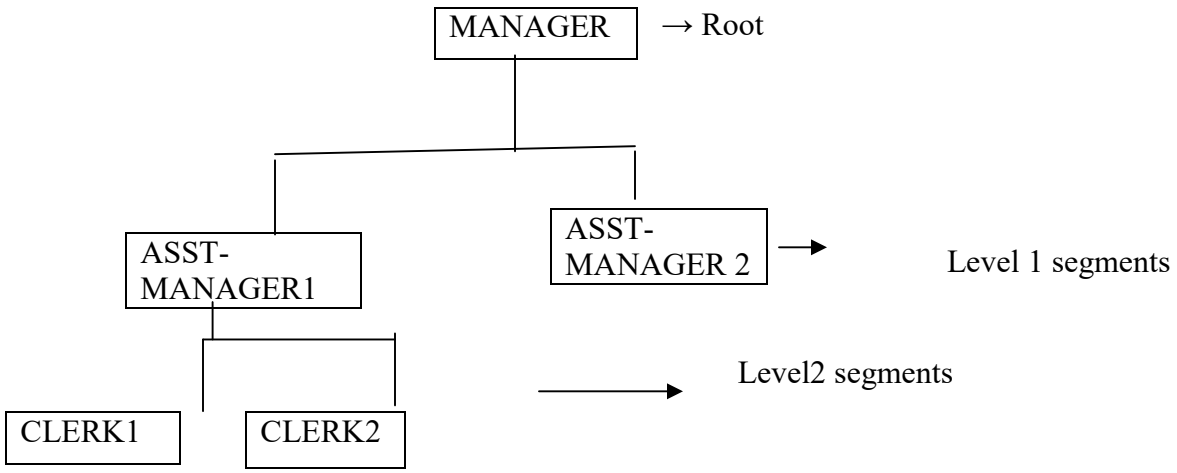
1. Relational Model 2. Network Model 3. Hierarchical Models.

Q.6. Explain hierarchical model? What are its advantages and disadvantages?

Ans. A hierarchical data model is a data model in which the data is organized into a tree-like structure. The structure represents information using parent/child relationships. Each parent can have many children but each child only has one parent. The hierarchical model was developed in the 1960's to manage large amounts of data. The hierarchical structure contains levels, or segments. The root segment is the parent of the level1 segments, which, in turn, are the parents of the levle2 segments, etc. in short; the hierarchical model depicts a set of one-to-many relationships between

a parent and its children segments. (Each parent can have many children, but each child has only one parent.)

Example: the following diagram is an example for hierarchical model



Advantages:

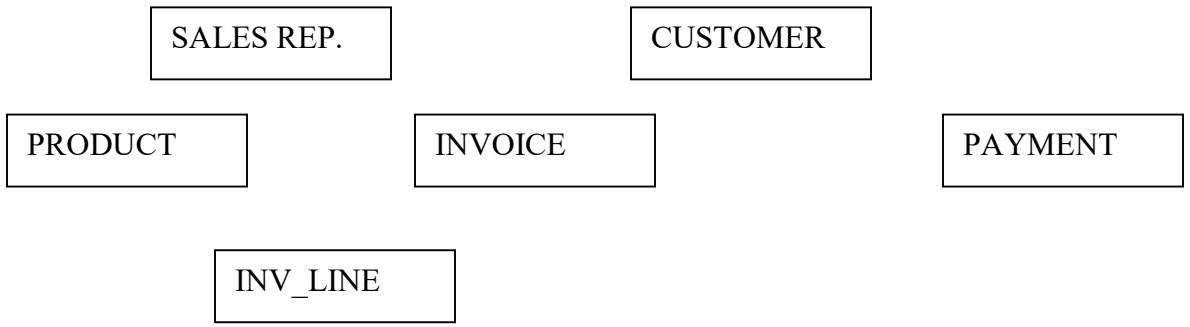
- Promotes data sharing
- Conceptual simplicity
- Database security
- Data independence
- Database integrity
- Efficient with 1: M relationship

Disadvantages:

- Complex implementation
- Difficult to manage
- Structural dependency
- Applications programming and use complexity
- No support of DDL and DML

Q.7. Explain network model? What are its advantages and Disadvantages?

Ans. In the network model, a parent can have several children and a child can also have many parent records. Records are physically linked through pointers. the network Model was created to represent complex relationships more effectively than the hierarchical model. This model was developed in the late 1960's. in the network model the user perceives the network database as a collection of records in 1:M relationships. In network database terminology, a relationship is called a set. Each set is composed of at least two record types: a parent record and a child record. A set represents a 1:M relationship between the parent and the child. **Example**: The following diagram is an example for network model



Advantages:

- Conceptual simplicity
- Handles more relationship types such as M:N
- Data access flexibility
- Promotes database integrity
- Match to standards
- Support of DDL and DML

Disadvantages:

- System complexity
- Structural dependency
- Simplex implementation.

Q.8. Explain relational model? What are its advantages and Disadvantages?

Ans. The relational model was introduced in 1970 by E.F codd. In the relational model, all data is maintained in the form of tables consisting of rows and columns. Data in two tables is related through common columns and not through pointers. This makes querying much easy in a relational DBMS. Unlike the hierarchical and network models, there are no physical links in relational model. The relationship type is often shown in a relational-schema. A table yields complete data and structural independence.

Example: the following set of tables-is an example for relational model

Table: **EMPLOYEE** table: **DEPARTMENT**

Empno.	Ename	Job	Salary	Deptno (F.K)
123	Ravi	Manager	12000	10
124	Rajesh	Clerk	2000	20
125	Amar	Clerk	2400	10
126	Kiran	Accountant	4000	30

→

Deptno P.K	Dept-name
10	Finance
20	Accounts
30	Marketing

Advantages:

- Structural independence
- Improved conceptual simplicity
- Easier database design, implementation, management, and use

Disadvantages:

- Requires large hardware and system software overhead
- Possibility of poor design and implementation

Characteristics of Relational Database

- Relational database systems have the following characteristics:
- The whole data is conceptually represented as an orderly arrangement of data into rows and columns, called a relation or table.
- All values are scalar. That is, at any given row/column position in the relation there is one and only one value.
- All operations are performed on an entire relation and result is an entire relation, a concept known as closure.

Q.9. What is DBMS? What are the functions of DBMS?

Ans. **"**A Data base is a collection of logically related data stored in a standardized format and is sharable by multiple users."

DBMS: Data Base management System is collection of database and a set of application programs that allow us to organize, manipulate and retrieve information.

Functions of DBMS: A Database Management System is some software that does the following functions:

- It manages the meta data in the data dictionary. "*Metadata is data about data."* It includes information about tables i.e their names, structures and relation between those.
- It allows us to orgaize the data properly. A DBMS arranges the data in chosen structure automatically.
- It automattically transforms input data as per the requiemnets of data dictionary
- It manages an environment to create and enforce securities: can create constrains, data validation rules, and define a range of values in DBMS
- It enforces data integrity at all levels as there will not be any anomolies in the DBMS.
- It supports query language to support creation, manipulation and retrievals.

Q.10. WHAT IS VIEW OF DATA IN DBMS? OR EXPLAIN DATA ABSTRACTION, INSTANCE AND SCHEMA IN DBMS

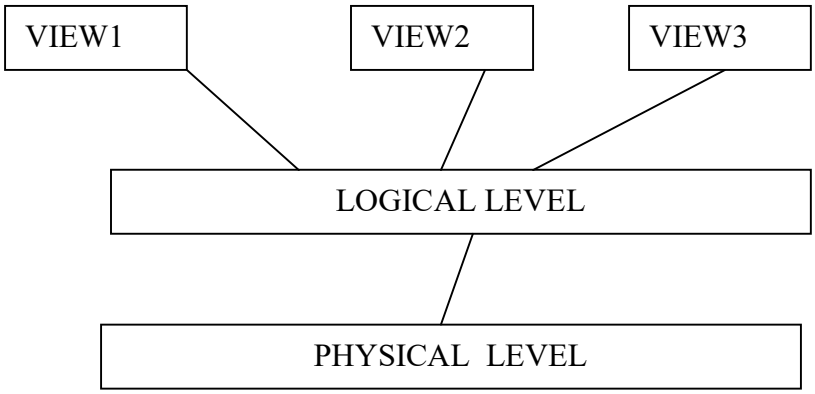
Ans. A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an ***abstract*** view of the data. That is, the system hides certain details of how the data are stored and maintained.

Abstraction is one of the main features of database systems. **Hiding irrelevant details from user** and providing abstract view of data to users, helps in easy and efficient **user-database** interaction. In the previous tutorial, we discussed the three level of DBMS architecture, The top level of that architecture is "view level". The view level provides the "**view of data**" to the users and hides the irrelevant details such as data relationship, database schema, constraints, security etc from the user.
To fully understand the view of data, you must have a basic knowledge of data abstraction and instance & schema. Refer these two tutorials to learn them in detail.

1. Data abstraction
2. Instance and schema

Data Abstraction in DBMS

Database systems are made-up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.

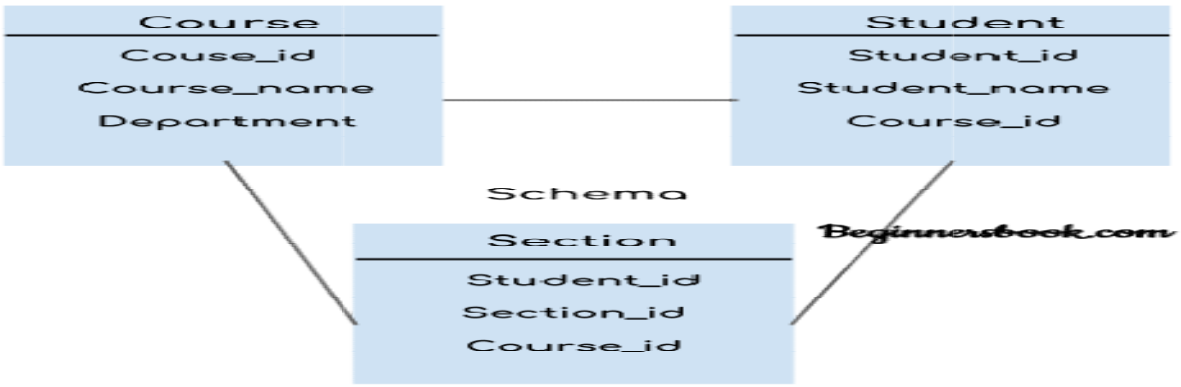


We have three levels of abstraction:
Physical level: This is the lowest level of data abstraction. It describes how data is actually stored in database. You can get the complex data structure details at this level.
Logical level: This is the middle level of 3-level data abstraction architecture. It describes what data is stored in database.
View level: Highest level of data abstraction. This level describes the user interaction with database system.
Example: Let's say we are storing customer information in a customer table. At **physical level** these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.
At the **logical level** these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented. The programmers generally work at this level because they are aware of such things about database systems.
At **view level**, user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored; such details are hidden from them.

DBMS Schema

Definition of schema: Design of a database is called the schema. Schema is of three types: Physical schema, logical schema and view schema.
For example: In the following diagram, we have a schema that shows the relationship between three tables: Course, Student and Section. The diagram only shows the design of the database, it doesn't show

the data present in those tables. Schema is only a structural view (design) of a database as shown in the diagram below.



The design of a database at physical level is called **physical schema**, how the data stored in blocks of storage is described at this level.
Design of database at logical level is called **logical schema**, programmers and database administrators work at this level, at this level data can be described as certain types of data records gets stored in data structures, however the internal details such as implementation of data structure is hidden at this level (available at physical level).
Design of database at view level is called **view schema**. This generally describes end user interaction with database systems.

DBMS Instance

Definition of instance: The data stored in database at a particular moment of time is called instance of database. Database schema defines the variable declarations in tables that belong to a particular database; the value of these variables at a moment of time is called the instance of that database.
For example, let's say we have a single table student in the database, today the table has 100 records, so today the instance of the database has 100 records. Let's say we are going to add another 100 records in this table by tomorrow so the instance of database tomorrow will have 200 records in table. In short, at a particular moment the data stored in database is called the instance that changes over time when we add or delete data from the database.

Q.11. WHAT IS DATABASE DESIGN? EXPLAIN

A properly designed database provides you with access to up-to-date, accurate information. Because a correct design is essential to achieving your goals in working with a database, investing the time required to learn the principles of good design makes sense. In the end, you are much more likely to end up with a database that meets your needs and can easily accommodate change.

Database design refers to the activities that focus on the design of the database structure that will be used to store and manage end-user data. To meet all user requirements the database structure must be designed carefully. In fact, database design is very important aspect of databases. Even a good DBMS will perform poorly with as badly designed database.

To achieve proper database design, the database designer needs to identify the expected use of the database correctly. A well designed database facilitates data management and generates accurate and valuable information. A poorly designed database may lead to bad decision making and bad decision making can lead to the failure of an organization.
Good database design should take care of the following.

- Avoiding data redundancy
- Maintaining data integrity

- Data backup and recovery facilities
- Data security
- Facilitating extensibility
- Efficient data storage

The design process

The design process consists of the following steps:

- **Determine the purpose of your database**
  This helps to prepare you for the remaining steps.
- **Find and organize the information required**
  Gather all the types of information you might want to record in the database, such as product name, order number and etc.
- **Divide the information into tables**
  Divide your information items into major entities or subjects, such as Products or Orders. Each subject then becomes a table.
- **Turn information items into columns**
  Decide what information you want to store in each table. Each item becomes a field, and is displayed as a column in the table. For example, an 'Employees' table might include fields such as Last Name and Hire Date.
- **Specify primary keys**
  Choose each table's primary key. The primary key is a column that is used to uniquely identify each row. An example might be Product ID or Order ID.
- **Set up the table relationships**
  Look at each table and decide how the data in one table is related to the data in other tables. Add fields to tables or create new tables to clarify the relationships, as necessary.
- **Refine your design**
  Analyze your design for errors. Create the tables and add a few records of sample data. See if you can get the results you want from your tables. Make adjustments to the design, as needed.
- **Apply the normalization rules**
  Apply the data normalization rules to see if your tables are structured correctly. Make adjustments to the tables, as needed.

Q.12. What is SQL? What are the types of SQL commands?

ANS. SQL stands for structured query language. SQL used to write queries that store, alter and retrieve data and metadata of a relational database.

Feature of SQL:

1. This is a Non-Procedural language
2. This is a unified language i.e. common for many databases
3. This is a Fourth Generation Language (4GL) in which programmers concentrate on *what* rather than *how.*

SQl Commands are four types:

1. Data Definition Language (DDL)
2. Data Manipulation Language (DML)
3. Data Control Language (DCL)
4. Transaction Control (TCL)

Data Definition Language (DDL) statements are used to define the database structure. These SQL statements define the structure of a database, including rows, columns, tables and database specifics such as file locations. DDL SQL statements are more part of the DBMS. Some examples:

- CREATE - to create objects in the database
- ALTER - alters the structure of the database
- DROP - delete objects from the database

- TRUNCATE - remove all records from a table, including all spaces allocated for the records are removed
- COMMENT - add comments to the data dictionary
- RENAME - rename an object

Data Manipulation Language (DML) statements are used for managing data within database objects. These SQL statements are used to retrieve and manipulate data. The most fundamental commands including DELETE, INSERT, SELECT, and UPDATE.

- SELECT - retrieve data from the a database
- INSERT - insert data into a table
- UPDATE - updates existing data within a table
- DELETE - deletes all records from a table, the space for the records remain.

Data Control Language (DCL) statements. These SQL statements control the security and permissions of the objects or parts of the database(s). DCL SQL statements are also more part of the DBMS.

- GRANT - gives user's access privileges to database
- REVOKE - withdraw access privileges given with the GRANT command

Transaction Control (TCL) statements are used to manage the changes made by DML statements. It allows statements to be grouped together into logical transactions.

- COMMIT - save work done
- SAVEPOINT - identify a point in a transaction to which you can later roll back
- ROLLBACK - restore database to original since the last COMMIT

Q.13. What is Data Storage and Querying in DBMS

Ans. A database system is partitioned into modules that deal with each of their responsibilities of the overall system. The functional components of a database system can be broadly divided into the storage manager and the query processor components.

1. **Data Storage:** The storage manager is important because databases typically require a large amount of storage space. Corporate databases range in size from hundreds of gigabytes to, for the largest databases, terabytes of data. A gigabyte is approximately 1000 megabytes (actually 1024) (1 billion bytes), and a terabyte is 1 million mega bytes (1 trillion bytes). Since the main memory of computers cannot store this much information, the information is stored on disks. Data are moved between disk storage and main memory as needed. Since the movement of data to and from disk is slow relative to the speed of the central processing unit. The storage manager is responsible for storing, retrieving, and updating data in the database. The storage manager components include:

- Authorization and integrity manager, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- Transaction manager, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.
- File manager, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- Buffer manager, which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory. The buffer manager is a critical part of the database system, since it enables the database to handle data sizes that are much larger than the size of main memory.

The storage manager implements several data structures as part of the physical system implementation:

- Data files, which store the database itself.
- Data dictionary, which stores metadata about the structure of the database, in particular the schema of the database.
- Indexes, which can provide fast access to data items. Like the index in a textbook, a database index provides pointers to those data items that hold a particular value. For example, we could

use an index to find the instructor record with a particular ID, or all instructor records with a particular name.

Hashing is an alternative to indexing that is faster in some but not all case. A query can usually be translated in to any of a number of alternative evaluation plans that all give the same result.

2. The Query Processor

The query processor is important because it helps the database system to simplify and facilitate access to data. The query processor allows database users to obtain good performance while being able to work at the view level and not be burdened with understanding the physical-level details of the implementation of the system. It is the job of the database system to translate updates and queries written in a nonprocedural language, at the logical level, into an efficient sequence of operations at the physical level.

The query processor components include:

• **DDL interpreter**, which interprets DDL statements and records the definitions in the data dictionary.

• **DML compiler**, which translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.

A query can usually be translated into any of a number of alternative evaluation plans that all give the same result. The DMLcompiler also performs **query optimization**; that is, it picks the lowest cost evaluation plan from among the alternatives.

• **Query evaluation engine**, which executes low-level instructions generated by the DML compiler.

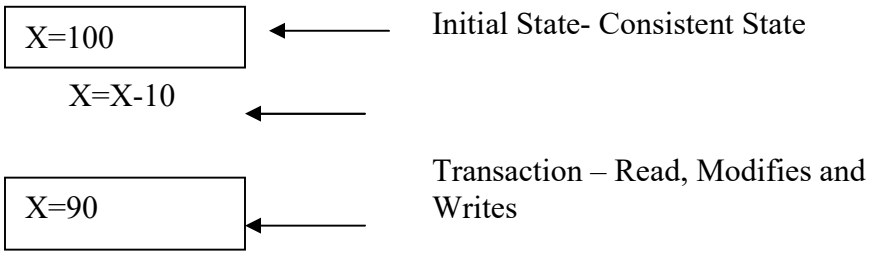
Q.14. Write notes on Transaction Processing. What are the Properties of the Transactions?

Ans. **Transaction Processing**: Transaction processing is the execution of database operations consisting of one or several requests as discrete units of work called transactions.

Transaction: Transaction is unit of work done.. It can be some action that reads from and /or writes to a database. A transaction must be either entirely completed or aborted. Most real-world database transactions are formed by two or more database requests. A database request is the equivalent of a single SQL statement in a program. A transaction can be:

- Simple SELECT statement to the list table contents
- A series of related UPDATE statements to change the values of attributes
- A combination of SELECT, UPDATE, and INSERT statements.

Exmaple: Following is an example of a transaction. A transaction that changes the contents of the database must alter the database from one consistent database state to another. And , every transaction must begin with the database in a known consistent state.



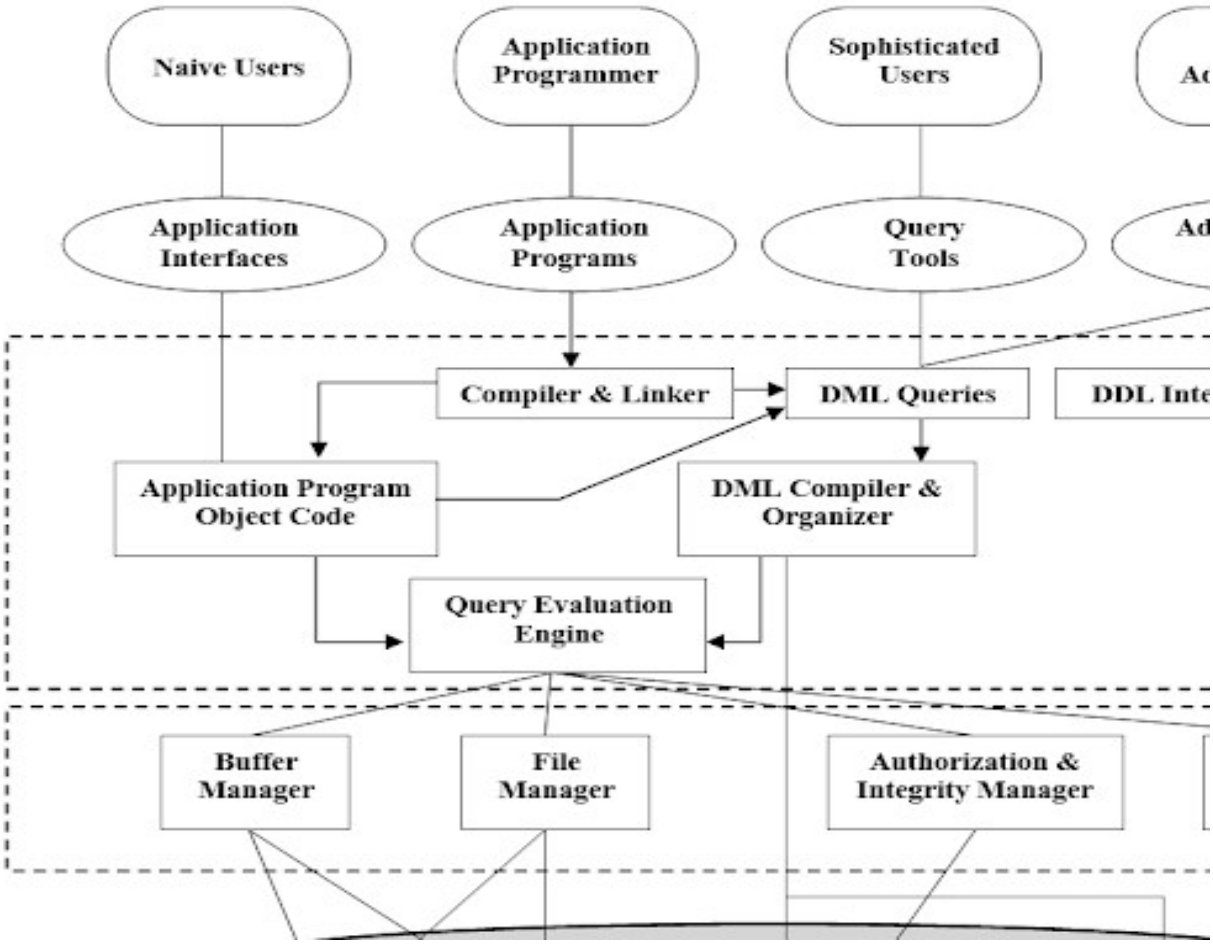
Properties of Transactions: Each transaction must display ACID properties: Atomicity, Consistency, Isolation and Durability.

- **Atomicity**: Atomicity requires that all operations of a transaction be completed; if not the transaction is aborted
- **Consistency**: Consistency, meaning any database constraints that must be true before the transaction must also be true after the transaction.
- **Isolation**: it means that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed.
- **Durability**: It menas that the changes are permanent.

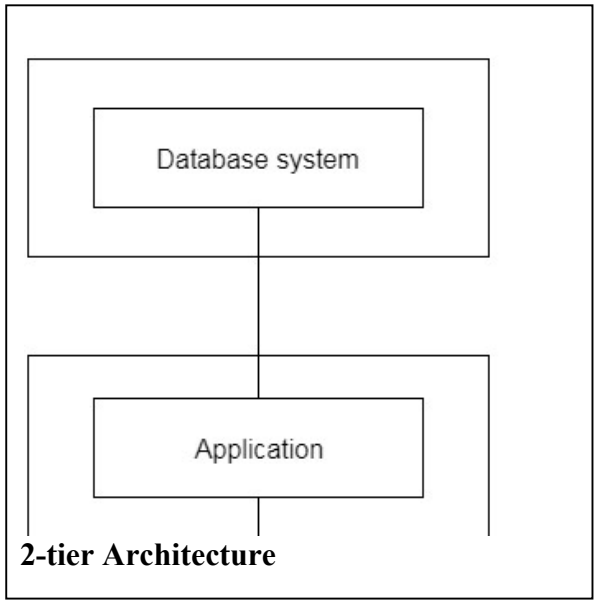
Q.15. Explain about Database Architecture

We are now in a position to provide a single picture of the various components of a database system and the connections among them.
The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

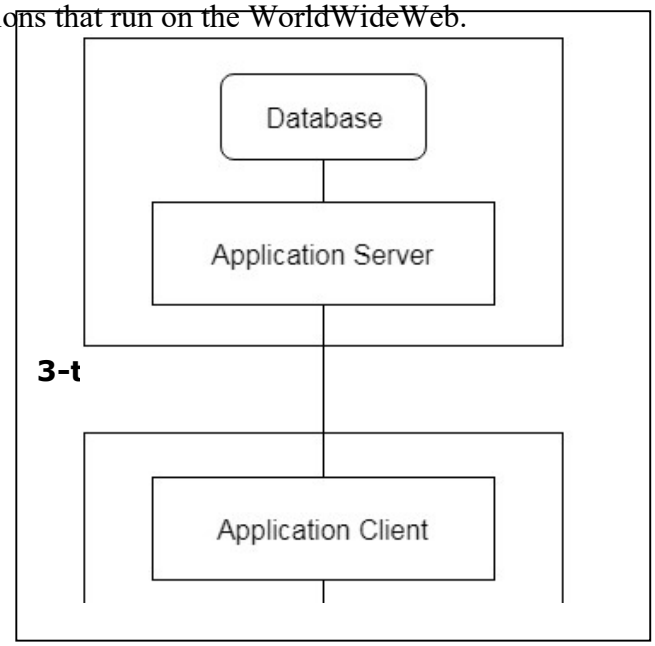
Most users of a database system today are not present at the site of the database system, but connect to it through a network. We can therefore differentiate between **client** machines, on which remote database users work, and **server** machines, on which the database system runs.



Database applications are usually partitioned into two or three parts. In a **two-tier architecture**, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements. Application program interface standards like ODBC and JDBC are used for interaction between the client and the server. In contrast, in a **three-tier architecture**, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an **application server**, usually through a forms interface.
The application server in turn communicates with a database system to access data. The **business logic** of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients. Three-tier applications are more appropriate for large applications, and for applications that run on the WorldWideWeb.



2-tier Architecture



3-t

Q.16. Explain different types of Database Users and Administrators

Ans. A primary goal of a database system is to retrieve information from and store new information in the database. People who work with a database can be categorized as database users or database administrators.

1) Database Users and User Interfaces

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for the different types of users:

• **Naive users** are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously. For example, a bank teller who needs to transfer $50 from account A to account B invokes a program called transfer. This program asks the teller for the amount of money to be transferred, the account from which the money is to be transferred, and the account to which the money is to be transferred.

As another example, consider a user who wishes to find her account balance over the World Wide Web. Such a user may access a form, where she enters her account number. An application program at the

Web server then retrieves the account balance, using the given account number, and passes this information back to the user.
The typical user interface for naive users is a forms interface, where the user can fill in appropriate fields of the form. Naive users may also simply read reports generated from the database.
•**Application programmers** are computer professionals who write application programs. Application programmers can choose from many tools to develop user interfaces. Rapid application development (RAD) tools are tools that enable an application programmer to construct forms and reports without writing a program. There are also special types of programming languages that combine imperative control structures (for example, for loops, while loops and if-then-else statements) with statements of the data manipulation language. These languages, sometimes called fourth-generation languages, often include special features to facilitate the generation of forms and the display of data on the screen. Most major commercial database systems include a fourth generation language.
•**Sophisticated users interact** with the system without writing programs. Instead, they form their requests in a database query language. They submit each such query to a query processor, whose function is to break down DML statements into instructions that the storage manager understands. Analysts who submit queries to explore data in the database fall in this category.
•**Specialized users** are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework. Among these applications are computer-aided design systems, knowledge base and expert systems, systems that store data with complex data types (for example, graphics data and audio data), and environment-modelling systems.

2 Database Administrators

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data. A person who has such central control over the system is called a database administrator (DBA). The functions of a DBA include:
•**Schema definition.** The DBA creates the original database schema by executing a set of data definition statements in the DDL.
•**Schema and physical-organization modification.** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.
•**Granting of authorization for data access.** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.

•**Routine maintenance.**
Examples of the database administrator's routine maintenance activities are:
a) Periodically backing up the database, either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
b) Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
c) Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.
DBA Responsibilities

- Installation, configuration and upgrading of Microsoft SQL Server/MySQL/Oracle server software and related products.
- Evaluate MSSQL/MySQL/Oracle features and MSSQL/MySQL/Oracle related products.
- Establish and maintain sound backup and recovery policies and procedures.
- Take care of the Database design and implementation.

- Implement and maintain database security (create and maintain users and roles, assign privileges).
- Database tuning and performance monitoring.
- Application tuning and performance monitoring.
- Setup and maintain documentation and standards.
- Plan growth and changes (capacity planning).
- Work as part of a team and provide 7×24 supports when required.
- Do general technical trouble shooting and give consultation to development teams.
- Interface with MSSQL/MySQL/Oracle for technical support.

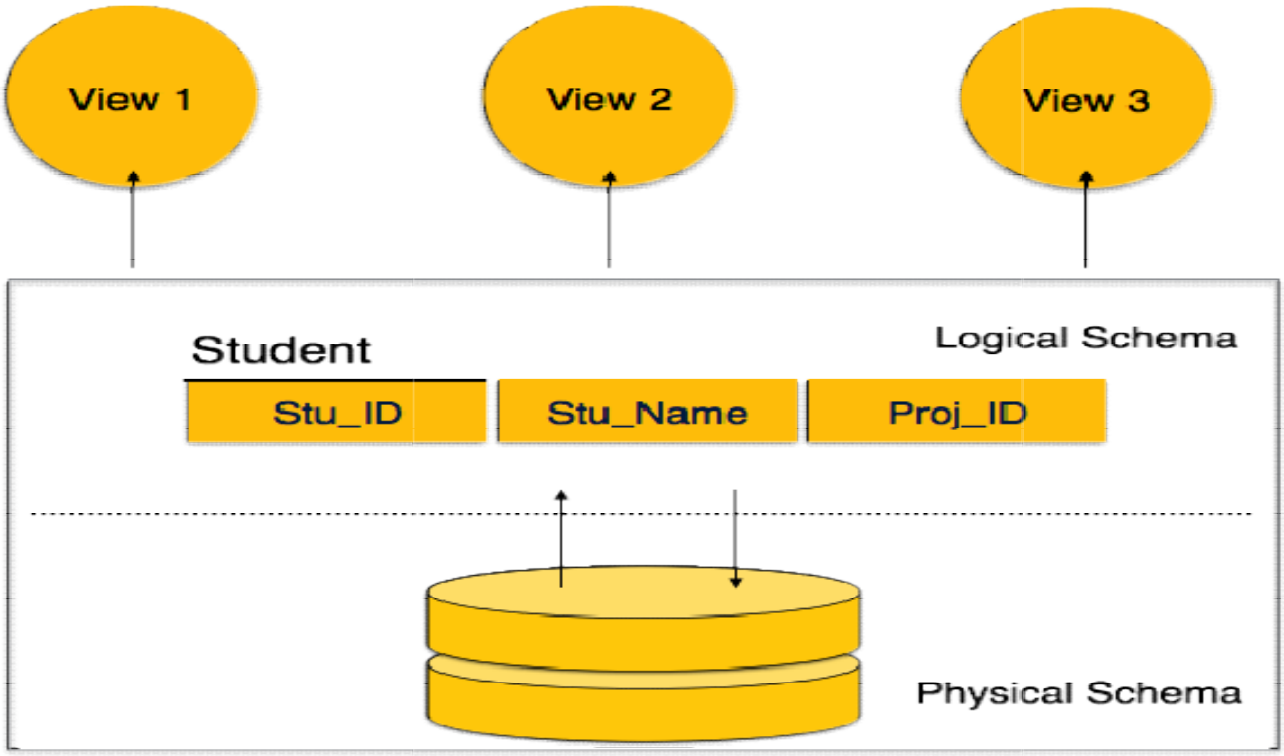
Types of DBA

1. **Administrative DBA** – Work on maintaining the server and keeping it running. Concerned with backups, security, patches, replication, etc. Things that concern the actual server software.
2. **Development DBA** – works on building queries, stored procedures, etc. that meet business needs. This is the equivalent of the programmer.
3. **Architect** – Design schemas. Build tables, FKs, PKs, etc. Work to build a structure that meets the business needs in general. The design is then used by developers and development DBAs to implement the actual application.
4. **Data Warehouse DBA** – Newer role, but responsible for merging data from multiple sources into a data warehouse. May have to design warehouse, but cleans, standardizes, and scrubs data before loading.
5. **Application DBA-** Application DBAs straddle the fence between the DBMS and the application software and are responsible for ensuring that the application is fully optimized for the database and vice versa. They usually manage all the application components that interact with the database and carry out activities such as application installation and patching, application upgrades, database cloning, building and running data cleanup routines, data load process management, etc

Q. 17. What is Database Schema

ANS. A database schema is the skeleton structure that represents the logical view of the entire database. It defines how the data is organized and how the relations among them are associated. It formulates all the constraints that are to be applied on the data.

A database schema defines its entities and the relationship among them. It contains a descriptive detail of the database, which can be depicted by means of schema diagrams. It's the database designers who design the schema to help programmers understand the database and make it useful.



A database schema can be divided broadly into two categories −

- **Physical Database Schema** − This schema pertains to the actual storage of data and its form of storage like files, indices, etc. It defines how the data will be stored in a secondary storage.
- **Logical Database Schema** − This schema defines all the logical constraints that need to be applied on the data stored. It defines tables, views, and integrity constraints.

Q. 18. WHAT IS KEY? EXPLAIN DIFFERENT TYPES OF KEYS.

ANS. Keys are very important part of Relational database model. They are used to establish and identify relationships between tables and also to uniquely identify any record or row of data inside a table.

A Key can be a single attribute or a group of attributes, where the combination may act as a key.

Need a Key:

In real world applications, number of tables required for storing the data is huge, and the different tables are related to each other as well.

Also, tables store a lot of data in them. Tables generally extends to thousands of records stored in them, unsorted and unorganised.

Now to fetch any particular record from such dataset, you will have to apply some conditions, but what if there is duplicate data present and every time you try to fetch some data by applying certain condition, you get the wrong data. How many trials before you get the right data?

To avoid all this, **Keys** are defined to easily identify any row of data in a table.

Let's try to understand about all the keys using a simple example.

student_id	name	phone	age
1	Akon	9876723452	17
2	Akon	9991165674	19
3	Bkon	7898756543	18
4	Ckon	8987867898	19
5	Dkon	9990080080	17

Let's take a simple **Student** table, with fields student_id, name, phone and age.

Super Key:

Super Key is defined as a set of attributes within a table that can uniquely identify each record within a table. Super Key is a superset of Candidate key.
In the table defined above super key would include student_id, (student_id, name), phone etc.
Confused? The first one is pretty simple as student_id is unique for every row of data, hence it can be used to identity each row uniquely.
Next comes, (student_id, name), now name of two students can be same, but their student_id can't be same hence this combination can also be a key.
Similarly, phone number for every student will be unique, hence again, phone can also be a key.
So they all are super keys.

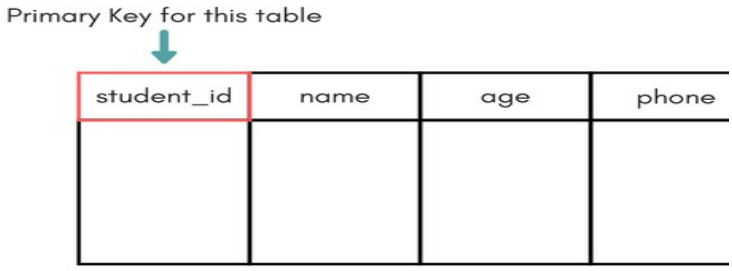
Candidate Key:

Candidate keys are defined as the minimal set of fields which can uniquely identify each record in a table. It is an attribute or a set of attributes that can act as a Primary Key for a table to uniquely identify each record in that table. There can be more than one candidate key.
In our example, student_id and phone both are candidate keys for table **Student**.

- A candiate key can never be NULL or empty. And its value should be unique.
- There can be more than one candidate keys for a table.
- A candidate key can be a combination of more than one columns(attributes).

Primary Key:

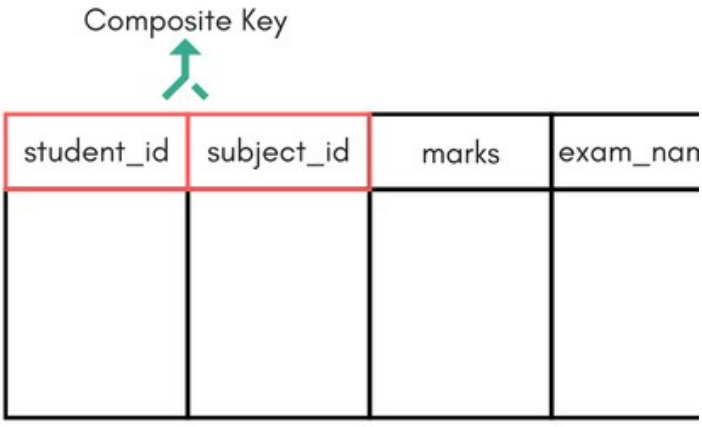
Primary key is a candidate key that is most appropriate to become the main key for any table. It is a key that can uniquely identify each record in a table.



For the table **Student** we can make the student_id column as the primary key.

Composite Key:

Key that consists of two or more attributes that uniquely identify any record in a table is called **Composite key**. But the attributes which together form the **Composite key** are not a key independentely or individually.



In the above picture we have a **Score** table which stores the marks scored by a student in a particular subject. In this table student_id and subject_id together will form the primary key, hence it is a composite key.

Secondary or Alternative key
The candidate key which are not selected as primary key are known as secondary keys or alternative keys.

Non-key Attributes
Non-key attributes are the attributes or fields of a table, other than **candidate key** attributes/fields in a table.

Non-prime Attributes
Non-prime Attributes are attributes other than **Primary Key attribute(s).**.

Q.19. WHAT IS SCEMA DIAGRAM? EXPLAIN WITH EXAMPLE.

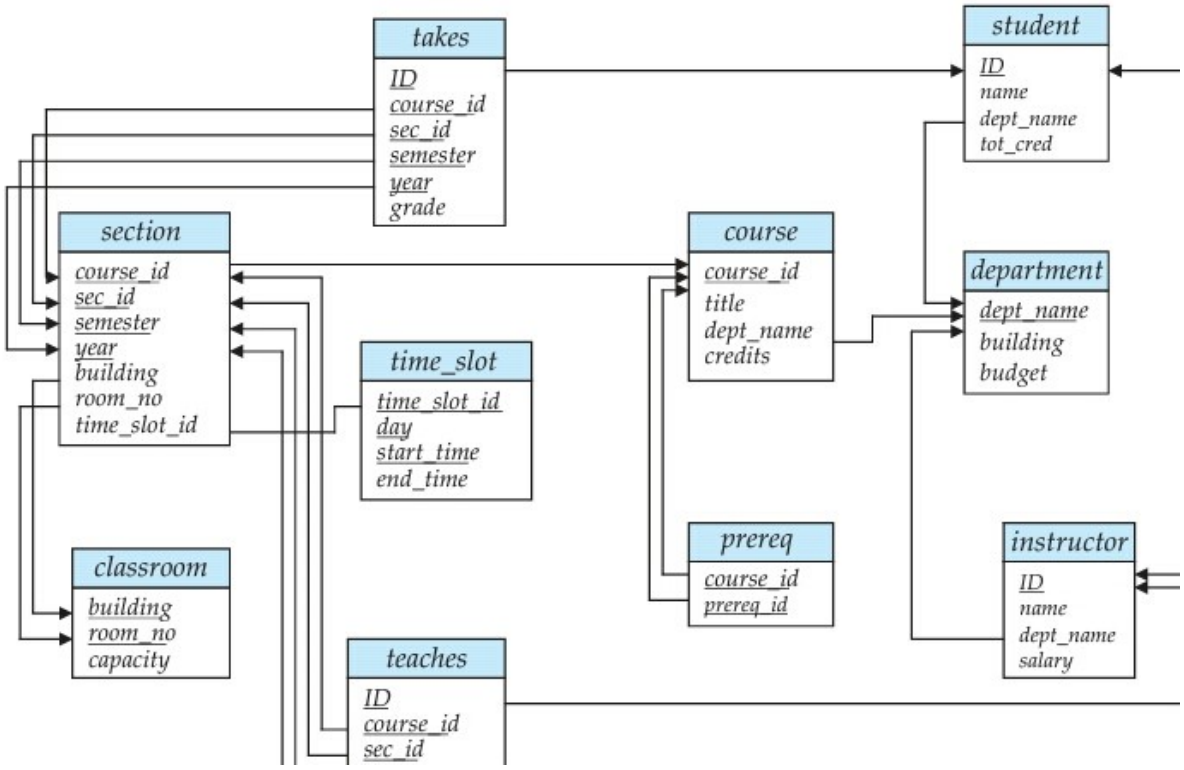
Ans. A **schema diagram** in a Database Management System (DBMS) is a visual representation or blueprint of the database's structure, defining the tables (relations), attributes (columns), primary and foreign keys, and the relationships between them. It acts as the "skeleton" or formal organization of the database, but does not contain any actual data.

Key Concepts of a Schema Diagram

- **Blueprint:** The diagram visually outlines the architecture of the database, making it easier for database designers, programmers, and administrators to understand and manage.
- **Logical View:** It primarily represents the logical schema, showing how data is organized and related from a high level, rather than how it is physically stored on disk.
- **Components:** The diagram typically includes:
  - **Tables/Relations:** Named boxes representing entities (e.g., STUDENT, COURSE).
  - **Attributes:** Column names within each table (e.g., StudentID, Name, CourseID).
  - **Keys:** Indicators for primary keys (unique identifiers for a row) and foreign keys (links to primary keys in other tables).
  - **Relationships:** Lines or arrows connecting tables to illustrate how data elements interact (e.g., a student enrolls in a course).

A database schema, along with primary key and foreign key dependencies, can be depicted by **schema diagrams**. Below diagram is the schema diagram for our university organization. Each relation appears as a box, with the relation name at the top in blue, and the attributes listed inside the box. Primary key attributes are shown underlined. Foreign key dependencies appear as arrows from the

foreign key attributes of the referencing relation to the primary key of the referenced relation.



Referential integrity constraints other than foreign key constraints are not shown explicitly in schema diagrams. Entity relationship diagrams let us represent several kinds of constraints, including general referential integrity constraints. Many database systems provide design tools with a graphical user interface for creating schema diagrams.

Q.20. Explain various Relational Algebra Operations. (OR) Explain Relational Set Operators.

Ans. To manipulate relations, relational model supports nine relational-algebra operations: SELECT, PROJECT, JOIN, INSERSECT, UNION, DIFFERENCE (MINUS), PRODUCT, DIVIDE and ASSIGNMENT.

Relational Algebra is a collection of operations to manipulate relations. Each operation takes one or more relations as its operands and produce another relation as its results. Union combines all rows from two tables, excluding duplicate rows (the data-type should match

(i) UNION (U): The union of two relations A and B is done by UNION command as:

A UNION B

It is the set of all types belonging to either A or B or both. Let us consider set A and B as:

Table A:

Rn	Nar
101	Am
102	Am
103	Vicl
104	Raj
110	Joyt

TABLE B:

Rn	Nar
103	Vic
106	Jimı
107	Ruc
110	Joyt

Query: SELECT RN, NAME FROM T1 UNION SELECT RN, NAME FROM T2;

R: A ∪ B

Rn	Na
101	An
103	Vic
102	An
104	Raj
106	Jim
107	Ru

(ii) Intersection (∩): The intersection operation selects the common tuples from the two relations. It is denoted by the symbol ∩. The intersection of two relations A and B is defined as:

A INTERSECT B

For example, if A and B are two sets, then intersection between these two are in R1.

Query: SELECT RN, NAME FROM T1 INTERSECT SELECT RN, NAME FROM T2;

R : A ∩ B

Rn	N
103	V
104	R

(iii) Difference (—): The difference operation removes common tuples from t first relation. The difference between two relations A and B be defined as:

A MINUS B

It is the set of all tuples belonging to set A but not belonging to B. It is denoted by (-). We can represent it as A - B. For example, from the above said two A and B sets, the difference between A and B be represented as:

Query: SELECT RN, NAME FROM T1 minus SELECT RN, NAME FROM T2;

R : A - B

Rn	N
101	A
102	Ri

(iv) Product: Product yields all possible pairs of rows from two tables- also known as Cortesion product. Therefore, if one table has 3 rows and the other table has 2 rows, the product yields a list composed of 3x2=6 rows

TABLE T1:

Sname
Ramu
Raju
krishna

TABLE T2

Cname
C
C++

Query: SELECT SNAME, CNAME FROM T1,T2
RESULT:

Sname	Cname
Ramu	C
Ramu	C++
Raju	C
Raju	C++
Krishna	C
Krishna	C++

V. Select: Select, also knonwn as RESTRICT, yields values for all the rows, or rows found in a table that satisfy given conditions. In other words, SELECT yields a horizontal subset of a table.

Select * from emp where job='clerk'

Empno	Ename	Job
101	Ravi	Manager
102	Ramesh	Clerk
103	Arun	Clerk
104	Krishna	Accountant

Empno	ename	Job
102	Ramesh	Clerk
103	Arun	Clerk

VI. Project: Project yields all values for selected attributes.

Empno	Ename	Job
101	Ravi	Manager
102	Ramesh	Clerk
103	Arun	Clerk
104	Krishna	Accountant

Select ename, job from emp;

Ename	Job
Ravi	Manager
Ramesh	Clerk
Arun	Clerk
Krishna	Accountant

VII. Join: Join allows information to be combined from two or more tables. Join allows the useof independent tables linked by common attributes. A natural join links tables by selecting only the rows with common values in their common attributes.

STD

RNO	NAME	CID
1	Ravi	10
2	Ramesh	30
3	Arun	80

COURSE

CID	CNAME
10	C
30	ORACLE

Qeury: SELECT RNO, NAME, STD.CID, CID,CNAME FROM STD, COURSE WHERE STD.CID=COURSE.CID.

RNO	NAME	CID	CNAME
1	Ravi	10	C
2	Ramesh	30	Oracle

Types of joins are

1. Equi join
2. Non-equi join
3. Outer join

VIII. Divide: Divide uses one single column table (say, A) as the divisor and one 2-column table (say, B) as the dividend. The tables must have a common column. The output of the DIVIDE operation is a single column with the values of column from the divided table (B) rows where the value of the common column in both tables match.

C1	C2
A	5
A	9
B	5
B	9

Divide

C1
A
B

gives

C2
5
9

IX. Assignment: Assignment assigns the result set to a variable. The assignment operator is ':=' .
For example, if we want to assign the maximum salary of all the employees to a variable MAX_SAL, we write:
MAX_SAL:= SELECT MAX(SAL) FROM EMP;